



**Universidad Nacional
Autónoma de México**



Facultad de Ingeniería

**Laboratorio de Arquitectura y Organización
de Computadoras**

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 5

Construcción de máquinas de estados usando memorias:
direccionamiento implícito

Fecha de Entrega

09 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando direccionamiento de memorias con el método de direccionamiento implícito.

2. Desarrollo

El direccionamiento implícito aprovecha el orden consecutivo de los estados. Para cada estado presente N , una de las dos transiciones posibles no se almacena en la memoria, sino que se obtiene mediante un incrementador como $N + 1$. La otra transición, denominada liga P , sí se guarda en la palabra de control. Para utilizar este método, la asignación binaria debe cumplir la regla $N, N + 1, P$. Cuando la carta ASM original no la cumple, es necesario reasignar los códigos de estado.

La palabra almacenada también contiene el código de la entrada que se debe probar, el bit VF y dos campos de salida. El bit VF indica con qué valor de la entrada seleccionada se debe utilizar la liga explícita. Si el resultado de la prueba coincide con VF , el contador carga P , en caso contrario, avanza a $N + 1$. De esta manera, sólo se almacena una liga de tres bits en lugar de dos.

2.1. Carta ASM y asignación binaria

La carta ASM utilizada tiene seis estados, tres entradas de decisión y cuatro salidas. Las entradas A , B y C se codificaron como 00, 01 y 10; el código 11 se reservó para $QAUX$. En la implementación, $QAUX$ se conecta a cero y permite representar estados sin prueba externa.

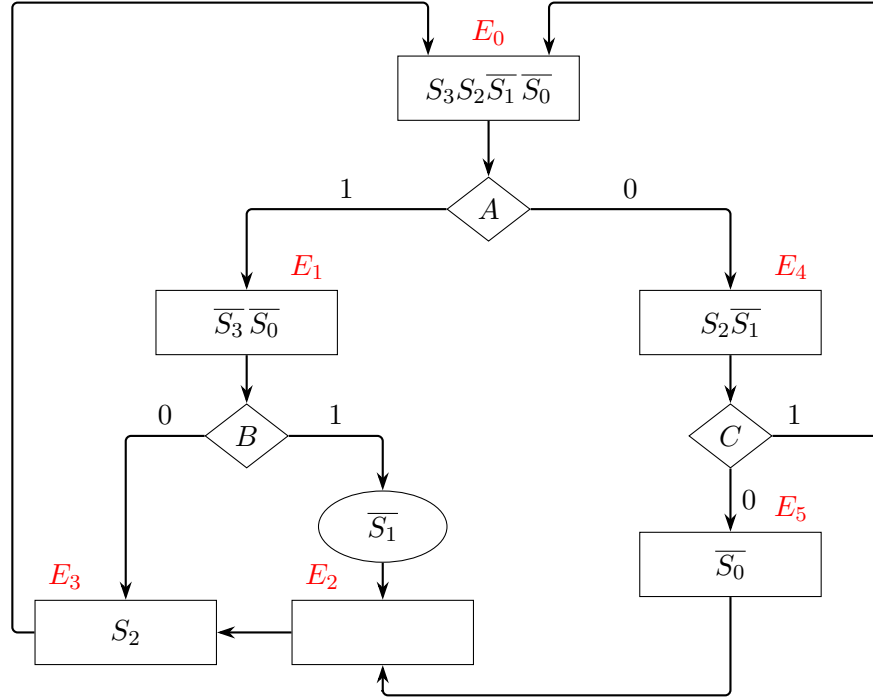


Figura 1: Carta ASM empleada utilizada para la práctica 5.

La asignación binaria seleccionada es:

Estado	E_0	E_1	E_2	E_3	E_4	E_5	Entrada	A	B	C	$QAUX$
$Q_2Q_1Q_0$	000	001	010	011	100	101	K_1K_0	00	01	10	11

Con esta codificación se obtienen las siguientes transiciones:

$$\begin{aligned}
 E_0 &: A = 0 \rightarrow E_4, \quad A = 1 \rightarrow E_1, \\
 E_1 &: B = 0 \rightarrow E_3, \quad B = 1 \rightarrow E_2, \\
 E_2 &: E_3, \quad E_3 : E_0, \\
 E_4 &: C = 0 \rightarrow E_5, \quad C = 1 \rightarrow E_0, \\
 E_5 &: E_2.
 \end{aligned}$$

Las transiciones $E_0 \rightarrow E_1$, $E_1 \rightarrow E_2$, $E_2 \rightarrow E_3$ y $E_4 \rightarrow E_5$ corresponden a $N+1$ y, por tanto, pueden generarse sin guardar una segunda liga.

2.2. Dimensionamiento de la memoria

Se requieren tres bits para representar los seis estados, por lo que la memoria dispone de $2^3 = 8$ direcciones. La palabra de control se organiza de la siguiente forma:

$$\underbrace{K_1 K_0}_{\text{prueba}} \quad \underbrace{VF}_{\text{valor de liga}} \quad \underbrace{L_2 L_1 L_0}_{\text{liga } P} \quad \underbrace{SF_3 SF_2 SF_1 SF_0}_{\text{salida falsa}} \quad \underbrace{SV_3 SV_2 SV_1 SV_0}_{\text{salida verdadera}}.$$

Por lo tanto, el número de bits por palabra es

$$m = 2 + 1 + 3 + 4 + 4 = 14 \text{ bits},$$

y el tamaño de la memoria es

$$2^n \times m = 2^3 \times 14 = 8 \times 14 = 112 \text{ bits}.$$

Aunque sólo se emplean seis estados, se programaron también las direcciones 110 y 111 con valores seguros para regresar a 000.

2.3. Tabla de direccionamiento y contenido de la memoria

La Tabla[1] muestra el programa grabado en la ROM. Donde K_1K_0 selecciona la entrada, VF determina cuándo se carga la liga explícita y $L_2L_1L_0$ contiene P . Los campos SF y SV se seleccionan cuando la entrada evaluada vale 0 o 1.

Tabla 1: Tabla de direccionamiento implícito y contenido de la ROM.

Estado	$Q_2Q_1Q_0$	K_1K_0	VF	Liga P	SF/SV
E_0	000	00	0	100	1100/1100
E_1	001	01	0	011	1010/1000
E_2	010	11	1	000	0011/0011
E_3	011	11	0	000	0111/0111
E_4	100	10	1	000	0101/0101
E_5	101	11	0	010	0010/0010
Recuperación	110	11	0	000	0011/0011
Recuperación	111	11	0	000	0011/0011

Por ejemplo, en E_0 la ROM entrega 00 0 100 1100 1100. El código 00 selecciona A . Si $A = 0$, su valor coincide con $VF = 0$ y se carga la liga explícita 100 si $A = 1$, no coincide con VF y el incrementador produce 001. En ambos casos la salida es 1100.

2.4. Arquitectura del sistema

El registro entrega el estado presente, que funciona como dirección de la ROM y como entrada del incrementador. La ROM genera los campos de prueba, VF , liga y salidas. El multiplexor de prueba selecciona A , B , C o $QAUX$ y produce $Qsel$. Después, una compuerta XNOR compara $Qsel$ con VF para generar la señal de carga:

$$\text{carga} = \overline{Qsel \oplus VF}.$$

Cuando $\text{carga}=1$, el multiplexor de control selecciona la liga P , cuando $\text{carga}=0$, selecciona $N + 1$. En paralelo, mux_salida escoge SF o SV directamente con $Qsel$.

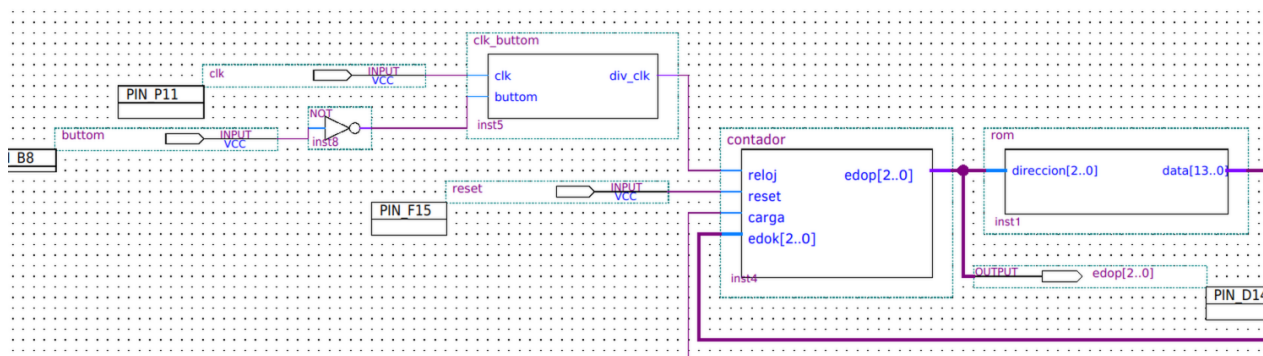


Figura 2: Conexión del reloj y del bloque contador dentro del sistema.

La Figura[3] muestra la estructura interna del contador con carga. La salida `edop` del registro se realimenta hacia el incrementador, que produce `edouno=edop+1`. El multiplexor `mux_control` recibe ese valor y la liga `edok` proveniente de la ROM. La señal `carga` decide cuál de los dos valores se aplica a `edos`, finalmente, el registro captura `edos` en el siguiente flanco ascendente de `reloj`. La entrada `reset` permite forzar el estado inicial 000.

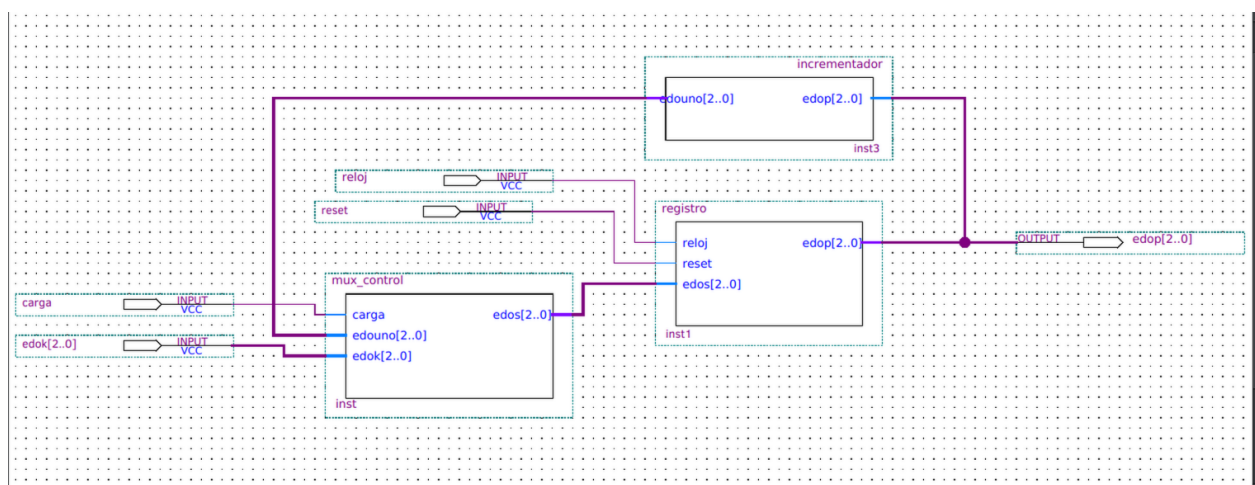


Figura 3: Diagrama interno del contador con carga: multiplexor, registro e incrementador.

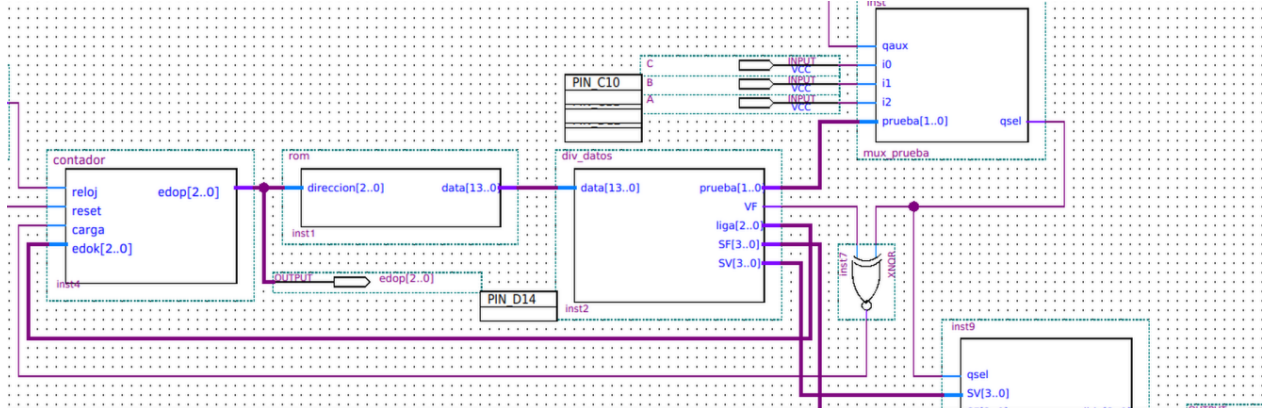


Figura 4: Conexión del contador, la ROM y la separación de la palabra de control.

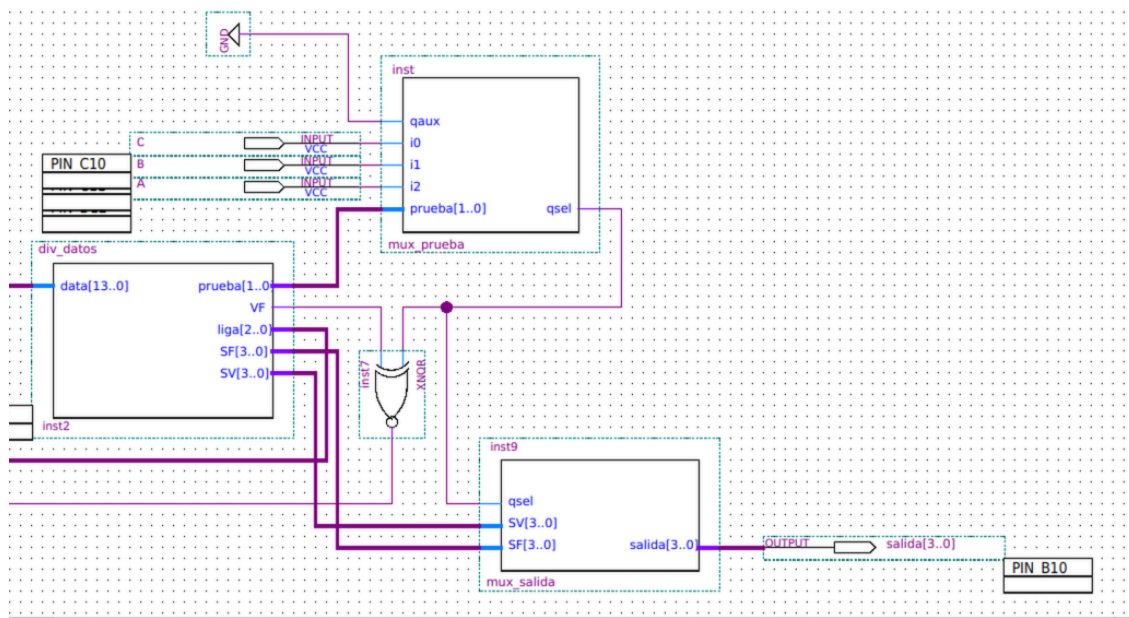


Figura 5: Selector de prueba y lógica de selección de las salidas.

2.5. Módulos implementados en VHDL

Registro e incrementador: El registro conserva el estado presente. Su reinicio es asíncrono y activo en bajo; con `reset='0'` fuerza 000, mientras que cada flanco ascendente carga el estado siguiente. El incrementador calcula permanentemente $N + 1$.

Listing 1: Registro del estado presente.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;

```

```
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity registro is
7     Port ( reloj : in std_logic;
8           reset  : in std_logic;
9           edos   : in std_logic_vector (2 downto 0);
10          edop    : out std_logic_vector (2 downto 0)
11        );
12 end registro;
13
14 architecture Behavioral of registro is
15
16 signal internal_value: std_logic_vector (2 downto 0) := B"000";
17 begin
18     process(reloj, reset, edos)
19     begin
20         if reset = '0' then
21             internal_value <= B"000";
22         elsif rising_edge(reloj) then
23             internal_value <= edos;
24         end if;
25     end process;
26
27     process(internal_value)
28     begin
29         edop <= internal_value;
30     end process;
31
32 end Behavioral;
```

Listing 2: Incrementador para obtener la dirección implícita $N + 1$.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity incrementador is
7     Port ( edop: in std_logic_vector (2 downto 0);
8           edouno : out std_logic_vector (2 downto 0)
9        );
10 end incrementador;
11
12 architecture Behavioral of incrementador is
13 begin
14
15     process(edop)
16     begin
17         edouno <= edop + 1;
18     end process;
19
20 end Behavioral;
```

Multiplexor de control: Este bloque forma el contador con carga: selecciona el valor incrementado cuando $carga=0$ o la liga almacenada cuando $carga=1$.

Listing 3: Selección entre $N + 1$ y la liga explícita P .

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity mux_control is
7      Port ( carga : in std_logic;
8            edouno : in std_logic_vector (2 downto 0);
9            edok  : in std_logic_vector (2 downto 0);
10           edos   : out std_logic_vector (2 downto 0)
11        );
12 end mux_control;
13
14 architecture Behavioral of mux_control is
15 begin
16
17     process(carga, edouno, edok)
18     begin
19
20         if carga = '0' then
21             edos <= edouno;
22         elsif carga = '1' then
23             edos <= edok;
24         end if;
25
26     end process;
27
28 end Behavioral;

```

ROM y división de datos. La ROM contiene ocho palabras de 14 bits. El bloque `div_datos` separa los bits [13 : 12] como prueba, el bit [11] como VF , los bits [10 : 8] como liga, [7 : 4] como salida falsa y [3 : 0] como salida verdadera.

Listing 4: ROM de 8×14 bits con el microprograma.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity rom is
7      Port (
8          direccion : in std_logic_vector(2 downto 0);
9          data       : out std_logic_vector(13 downto 0)
10     );
11 end rom;
12
13 architecture Behavioral of rom is
14
15     type mem is array (0 to 7) of std_logic_vector(13 downto 0);
16     signal internal_mem : mem;
17
18 begin
19
20     -----
21     -- FORMATO DE LA PALABRA
22     --
23     -- data(13 downto 12) = Prueba
24     -- data(11)           = VF

```

```

25  -- data(10 downto 8) = Liga
26  -- data(7 downto 4)  = Salida Falsa
27  -- data(3 downto 0)  = Salida Verdadera
28  --
29  -- Prueba:
30  -- 00 = A
31  -- 01 = B
32  -- 10 = C
33  -- 11 = QAUX / sin prueba
34  -----
35
36
37  -- ESTADO 0 = 000
38  -- Prueba A
39  -- A=0 -> Liga = 100 (Estado 4)
40  -- A=1 -> dirección implícita N+1 = 001 (Estado 1)
41  internal_mem(0) <=
42      "00" & "0" & "100" & "1100" & "1100";
43
44
45  -- ESTADO 1 = 001
46  -- Prueba B
47  -- B=0 -> Liga = 011 (Estado 3)
48  -- B=1 -> dirección implícita N+1 = 010 (Estado 2)
49  internal_mem(1) <=
50      "01" & "0" & "011" & "1010" & "1000";
51
52
53  -- ESTADO 2 = 010
54  -- Estado sin prueba
55  -- Continúa implícitamente a N+1 = 011 (Estado 3)
56  --
57  -- La liga es indiferente (***) en la tabla.
58  -- Guardamos 000 físicamente en la ROM.
59  internal_mem(2) <=
60      "11" & "1" & "000" & "0011" & "0011";
61
62
63  -- ESTADO 3 = 011
64  -- Sin prueba
65  -- Regresa explícitamente al Estado 0
66  internal_mem(3) <=
67      "11" & "0" & "000" & "0111" & "0111";
68
69
70  -- ESTADO 4 = 100
71  -- Prueba C
72  -- C=0 -> dirección implícita N+1 = 101 (Estado 5)
73  -- C=1 -> Liga = 000 (Estado 0)
74  internal_mem(4) <=
75      "10" & "1" & "000" & "0101" & "0101";
76
77
78  -- ESTADO 5 = 101
79  -- Sin prueba
80  -- Salta explícitamente al Estado 2 = 010
81  internal_mem(5) <=
82      "11" & "0" & "010" & "0010" & "0010";
83
84
85  -----
86  -- ESTADO 6 = 110
87  -- No utilizado en nuestra carta ASM.

```

```

88      -- Ponemos un valor seguro que regrese a Estado 0.
89      -----
90      internal_mem(6) <=
91          "11" & "0" & "000" & "0011" & "0011";
92
93
94      -----
95      -- ESTADO 7 = 111
96      -- Según la tabla:
97      -- prueba=11, VF=0, Liga=000
98      -----
99      internal_mem(7) <=
100          "11" & "0" & "000" & "0011" & "0011";
101
102
103      -----
104      -- LECTURA DE LA ROM
105      -----
106      process(direccion)
107      begin
108          data <= internal_mem(conv_integer(unsigned(direccion)));
109      end process;
110
111 end Behavioral;

```

Listing 5: Separación de los campos de la palabra de 14 bits.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity div_datos is
7      Port ( data : in std_logic_vector (13 downto 0);
8
9              prueba : out std_logic_vector (1 downto 0);
10             VF: out std_logic;
11             liga: out std_logic_vector (2 downto 0);
12             SF : out std_logic_vector (3 downto 0);
13             SV : out std_logic_vector (3 downto 0)
14         );
15 end div_datos;
16
17
18 architecture Behavioral of div_datos is
19 begin
20
21     process(data)
22     begin
23
24         prueba <= data(13 downto 12);
25         VF <= data(11);
26         liga <= data(10 downto 8);
27         SF <= data(7 downto 4);
28         SV <= data(3 downto 0);
29
30     end process;
31
32 end Behavioral;

```

Selección de prueba y salidas. El multiplexor de prueba convierte el código almacenado en la ROM en el valor de una de las entradas. El multiplexor de salida utiliza este mismo resultado para escoger el vector falso o verdadero.

Listing 6: Selección de *A*, *B*, *C* o *QAUX*.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity mux_prueba is
7      Port ( prueba : in std_logic_vector (1 downto 0);
8            i2: in std_logic;
9            i1: in std_logic;
10           i0: in std_logic;
11           qaux: in std_logic;
12           qsel: out std_logic
13      );
14 end mux_prueba;
15
16 architecture Behavioral of mux_prueba is
17 begin
18
19     process(prueba, i2, i1, i0, qaux)
20     begin
21
22         if prueba = "00" then
23             qsel <= i2;
24         elsif prueba = "01" then
25             qsel <= i1;
26         elsif prueba = "10" then
27             qsel <= i0;
28         elsif prueba = "11" then
29             qsel <= qaux;
30         end if;
31
32     end process;
33
34 end Behavioral;

```

Listing 7: Selección de la salida falsa o verdadera.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity mux_salida is
7      Port ( SF : in std_logic_vector (3 downto 0);
8            SV : in std_logic_vector (3 downto 0);
9            qsel: in std_logic;
10           salida : out std_logic_vector (3 downto 0)
11      );
12 end mux_salida;
13
14 architecture Behavioral of mux_salida is
15 begin
16
17     process(qsel, SF, SV)
18     begin
19

```

```

20     if qsel = '0' then
21         salida <= SF;
22     elsif qsel = '1' then
23         salida <= SV;
24     end if;
25
26     end process;
27
28 end Behavioral;

```

Reloj por botón. Para observar las transiciones en la tarjeta, el módulo `clk_button` genera un pulso controlado a partir del reloj de la FPGA y de un botón. Así se evita recorrer varios estados antes de poder identificar visualmente las salidas.

Listing 8: Acondicionamiento del reloj mediante botón.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clk_button is
7      port(
8          clk : in std_logic;
9          button : in std_logic;
10         div_clk : out std_logic
11     );
12 end clk_button;
13
14 architecture Behavioral of clk_button is
15
16     CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
17     CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
18     CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";
19
20     CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25 millones de
        pulsos
21
22     signal state: std_logic_vector(1 downto 0) := "00";
23     signal cuenta: std_logic_vector(27 downto 0) := X"0000000";
24
25 begin
26     process(clk, button)
27     begin
28         if rising_edge(clk) then
29             case state is
30                 WHEN STATE_START =>
31                     if button = '1' then
32                         state <= STATE_COUNT;
33                     end if;
34                 WHEN STATE_COUNT =>
35                     if cuenta >= MAX_CLK_COUNT then
36                         state <= STATE_DONE;
37                     end if;
38                 WHEN STATE_DONE =>
39                     state <= STATE_START;
40                 WHEN others =>
41                     state <= STATE_START;
42             end case;
43         end if;
44     end process;

```

```

45
46 process(clk)
47 begin
48     if rising_edge(clk) then
49         if state = STATE_COUNT then
50             cuenta <= cuenta + 1;
51         else
52             cuenta <= X"0000000";
53         end if;
54     end if;
55 end process;
56
57 process(state)
58 begin
59     if state = STATE_COUNT then
60         div_clk <= '1';
61     else
62         div_clk <= '0';
63     end if;
64 end process;
65 end Behavioral;

```

3. Simulación

La simulación en Multisim fue con una señal de reloj preestablecida, con **reset=1** y diferentes combinaciones para *A*, *B* y *C*. Las señales **edop** y **salida** permiten relacionar el estado presente con la palabra leída de la ROM. Como el registro cambia únicamente en el flanco ascendente, **edop** permanece estable entre flancos aunque las entradas se modifiquen.

En la Figura[6] se observan recorridos como 000→100→000 y 000→001→011→000. El primer recorrido ocurre cuando *A* = 0, por lo que se carga la liga de *E*₀ hacia *E*₄. El segundo comienza con *A* = 1, que selecciona la dirección implícita *N* + 1 = 001; posteriormente, con *B* = 0, se carga 011.

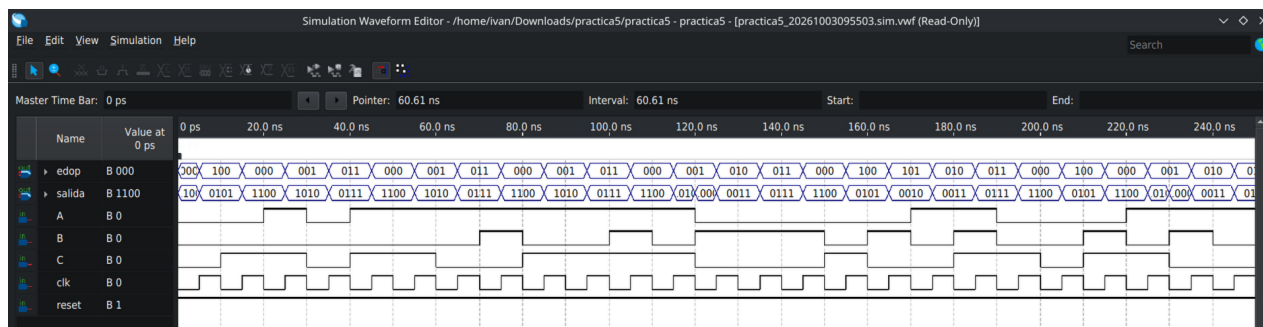


Figura 6: Primera parte de la simulación funcional del direccionamiento implícito.

La Figura[7] incluye secuencias que atraviesan la rama derecha de la carta ASM. Desde 100, si *C* = 0, no se carga la liga porque *C* ≠ *VF* y el contador avanza a 101, en *E*₅, *QAUX* = 0

coincide con $VF = 0$ y se carga la liga 010. Después, E_2 utiliza nuevamente el avance implícito para llegar a 011, y E_3 carga la liga 000.

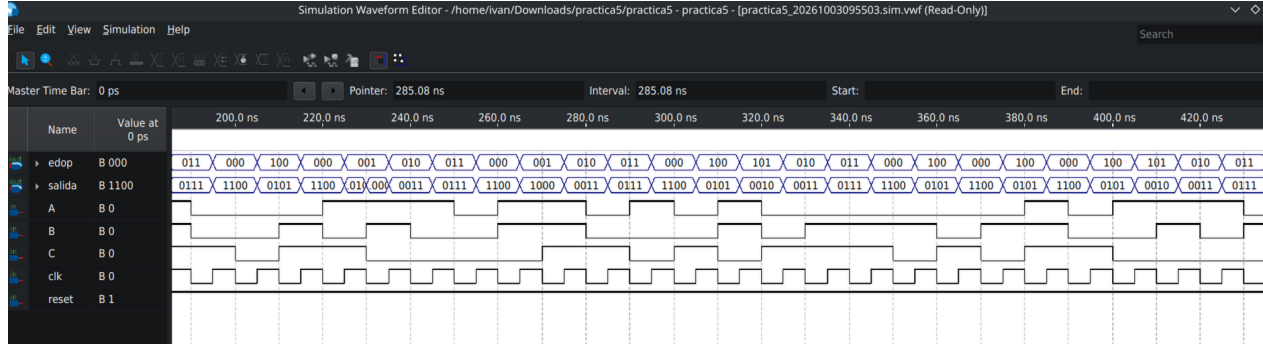


Figura 7: Transiciones mediante la liga explícita y la dirección implícita $N + 1$.

En la parte final, mostrada en la Figura[8], se repiten distintas rutas al modificar las entradas. Los valores de **salida** coinciden con el programa implementado: 1100 en E_0 , 1010 o 1000 en E_1 , 0011 en E_2 , 0111 en E_3 , 0101 en E_4 y 0010 en E_5 . Esto verifica tanto la selección de la liga como la selección de la salida correspondiente a cada rama.

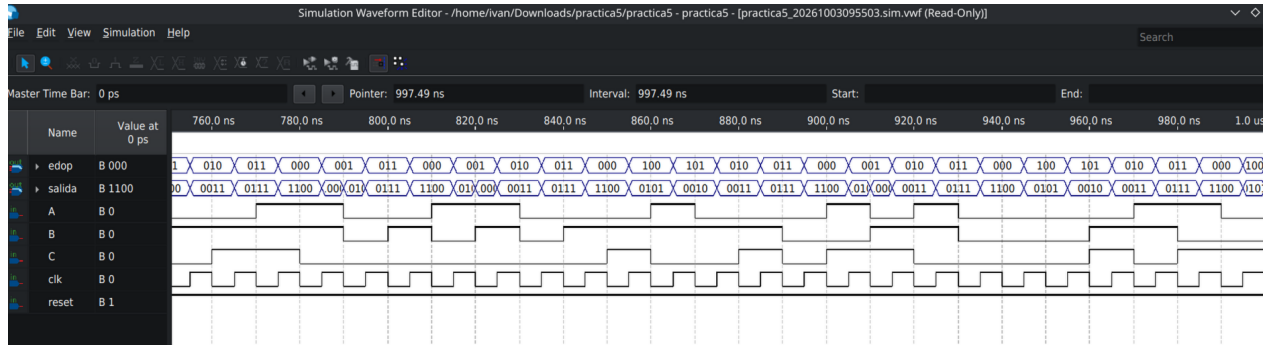
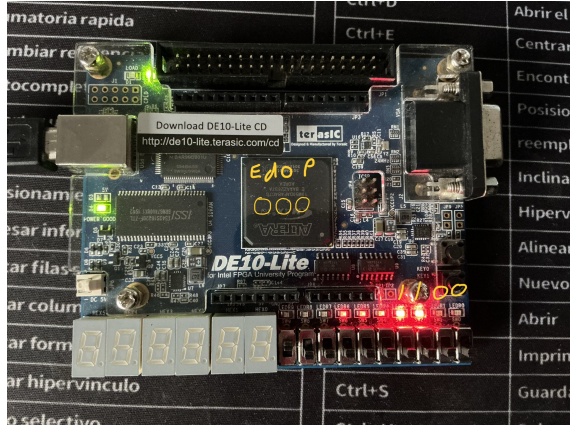


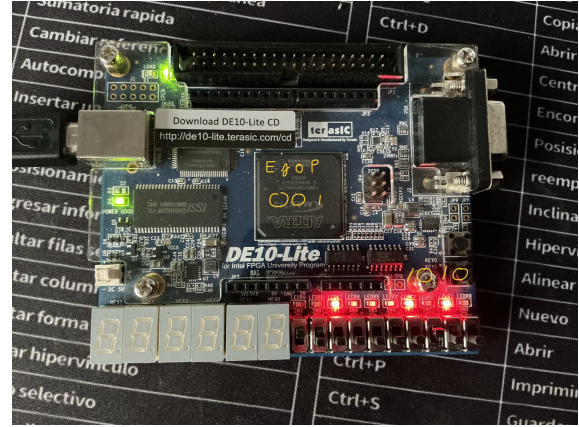
Figura 8: Parte final de la simulación, con diferentes recorridos de la carta ASM.

4. Resultados en la tarjeta FPGA

El diseño se implementó en la tarjeta de desarrollo DE10-Lite. Los tres LEDs asignados a **edop** muestran el código del estado presente y cuatro LEDs adicionales representan **salida**. Las fotografías permiten contrastar directamente los valores físicos con la Tabla 1.

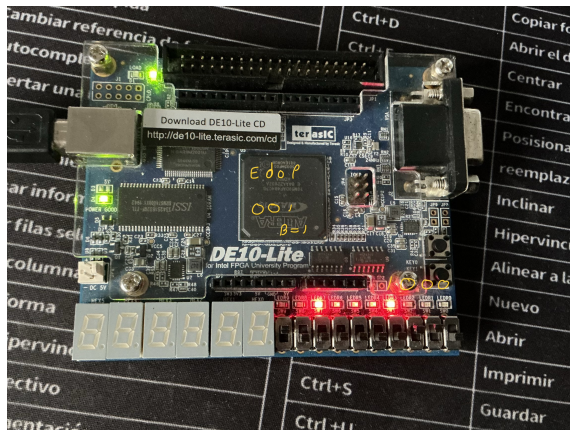


(a) $E_0 = 000$, salida 1100.

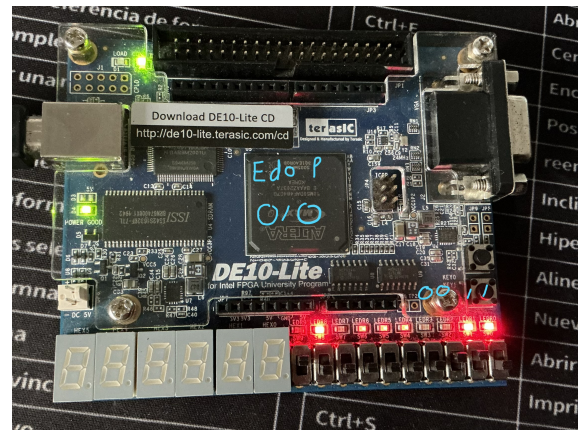


(b) $E_1 = 001$ con $B = 0$, salida 1010.

Figura 9: Comprobación física de E_0 y de la rama falsa de E_1 .

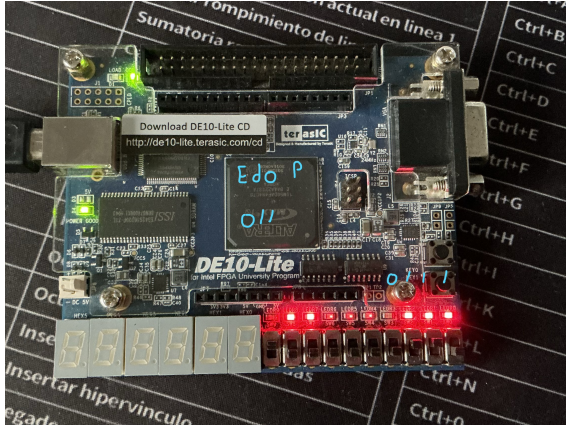


(a) $E_1 = 001$ con $B = 1$, salida 1000.

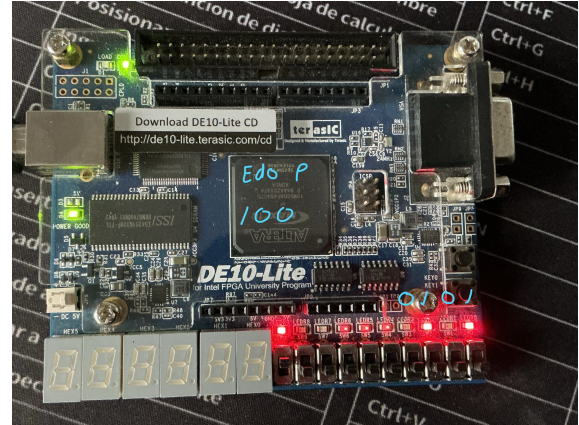


(b) $E_2 = 010$, salida 0011.

Figura 10: Rama verdadera de E_1 y evidencia del estado E_2 .



(a) $E_3 = 011$, salida 0111.



(b) $E_4 = 100$, salida 0101.

Figura 11: Comprobación física de los estados E_3 y E_4 .

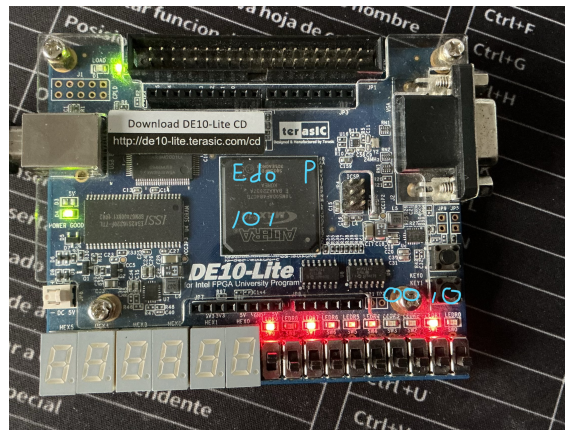


Figura 12: Estado $E_5 = 101$ con salida 0010.

Las implementación física confirmo que el contador avanza a $N + 1$ cuando la entrada seleccionada es diferente de VF y carga la liga almacenada cuando ambos valores coinciden. También se comprobó que el reinicio activo en bajo devuelve la máquina al estado 000 y que el pulso de reloj permite avanzar un estado a la vez.

5. Conclusión

Percival Ulises Espinoza Matamoros. A partir de la implementación de la máquina de estados con direccionamiento implícito, comprendí el funcionamiento del direccionamiento implícito, el cuál permite reducir el tamaño de la memoria al aprovechar la secuencia de estados siempre y cuando se cumpla la regla $N, N + 1, P$. La simulación y la implementación física me permitieron observar las transiciones de estado y la selección de salida, de forma que se logró implementar la carta ASM de manera correcta.

Oscar Iván Montiel Juárez. El desarrollo permitió relacionar la asignación binaria de estados con el funcionamiento del contador con carga. Se observó que la regla $N, N + 1, P$ es indispensable para aprovechar el direccionamiento implícito y que una asignación adecuada disminuye la memoria requerida. Las formas de onda mostraron claramente la diferencia entre incrementar el estado y cargar una dirección almacenada.

Amy Valentina Veraza García. La práctica ayudó a reforzar la relación entre la carta ASM, el microprograma y los módulos implementados en VHDL. Al probar el circuito en la DE10-Lite fue posible comprobar visualmente el estado presente y las salidas, así como el efecto de las entradas A, B y C sobre cada recorrido de la máquina.